



ICT306 Advanced Cybersecurity

WEEK 4 Hash, Digital Signature & Key Distribution

This lecture explores the foundational cryptographic mechanisms that underpin modern secure communications — from hash functions and digital signatures to the infrastructure that distributes and validates public keys.



Resources

Prescribed Reading

**Cryptography and Network Security
(Principles and Practice) William
Stallings — Chapter 13**

Recommended Reading

Whitman, Michael E., and Herbert J.
Mattord. *Principles of Information
Security*. Cengage Learning, 2021.

Module 9, Pages 1–26

Lecture Content

This lecture is structured around three major pillars of applied cryptography. Each topic builds on the previous, culminating in a complete picture of how trust and authenticity are established in modern networks.

01

Hash Functions

Properties, cryptographic requirements, SHA family, MD5/MD6, and OpenSSL demonstrations.

02

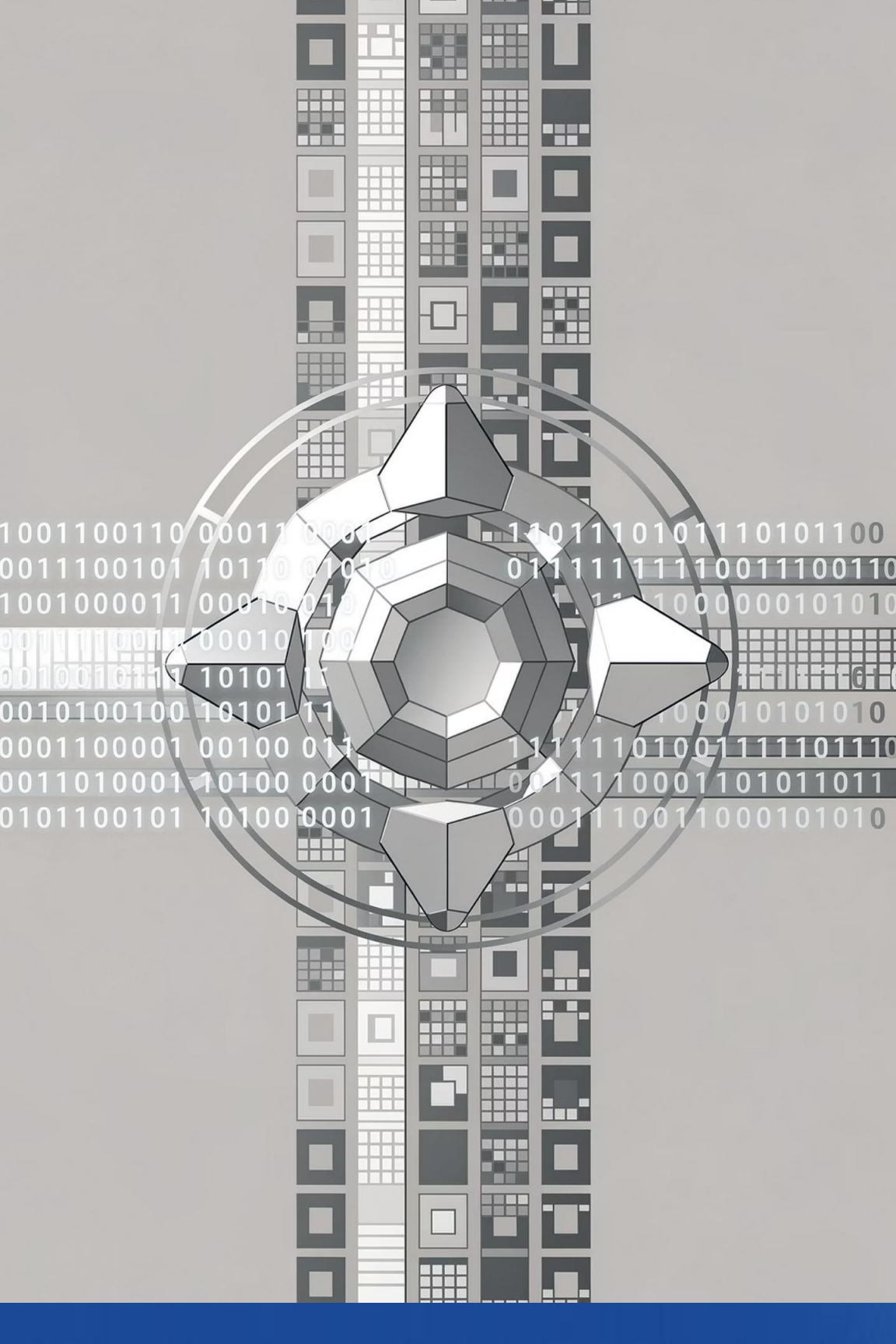
Digital Signatures

Overview, requirements, DSS/DSA standard, RSA vs. DSA schemes, and practical signing with OpenSSL.

03

Key Distribution

Symmetric and asymmetric key distribution, public key certificates, X.509, PKIX, CA hierarchy, and certificate validation.



Hash Functions

A hash function **H** accepts a variable-length block of data **M** and produces a fixed-size output **h = H(M)**. A well-designed hash function distributes outputs evenly and pseudo-randomly, and any change to even a single bit of input causes a significant change in the hash output.

Deterministic

The same message always produces the same hash value — no randomness at output.

One-Way

Computationally infeasible to derive **M** from **H(M)** — the pre-image resistance property.

Collision-Resistant

Infeasible to find two distinct inputs **M₁ ≠ M₂** such that **H(M₁) = H(M₂)**.

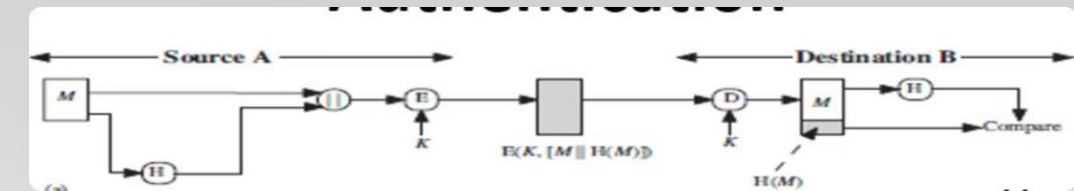
Avalanche Effect

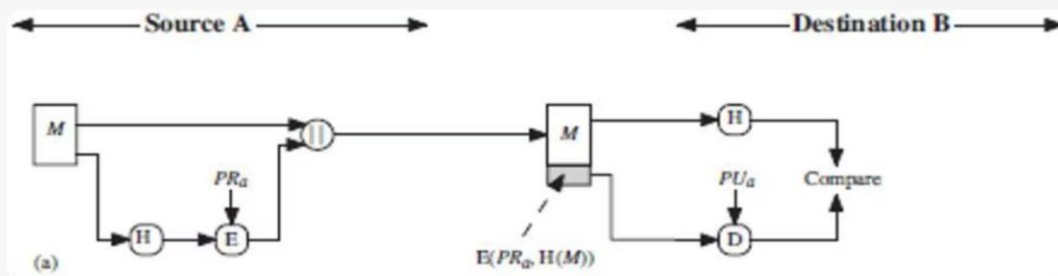
A small change in **M** causes a dramatically different hash value — ensuring sensitivity to input changes.

Application: Message Authentication

Message Authentication Codes (MACs) use hash functions to verify that a message has not been modified in transit — protecting against insertion, deletion, and replay attacks. When a hash function is used for this purpose, the resulting output is called a **message digest**.

- The sender computes a hash of the message and encrypts it with a shared key. The receiver decrypts and recomputes the hash — if they match, integrity is confirmed.





Application: Digital Signature

Rather than encrypting the entire document, a sender computes a hash of the document and encrypts only the hash using their private key. This dramatically reduces computational cost while still binding the sender's identity to the content. The receiver can verify authenticity by decrypting the hash and comparing it to a freshly computed hash of the received message.

Other Applications of Hash Functions

Beyond message authentication and digital signatures, cryptographic hash functions are deployed across many layers of system security.



One-Way Password Files

Operating systems store password hashes, not plaintext passwords. Authentication compares stored hashes — widely used in Linux, Windows, and macOS.



Intrusion & Virus Detection

File integrity monitoring tools hash system files at baseline and alert when hashes change unexpectedly, detecting unauthorized modifications or malware.



PRF & PRNG Construction

Hash functions are used to build Pseudorandom Functions (PRFs) and Pseudorandom Number Generators (PRNGs) used in key derivation and cryptographic protocols.

Secure Hashing Algorithm (SHA) Family

SHA algorithms generate cryptographically secure, one-way hash functions. Published by NIST as part of the Federal Information Processing Standard (FIPS), the SHA family has evolved to provide increasing digest lengths and stronger security guarantees.

- ❏ SHA-1 is now considered weak against collision attacks and has been deprecated. SHA-256 and SHA-512 are currently recommended for production use.

Table 11.3 Comparison of SHA Parameters

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message Digest Size	160	224	256	384	512
Message Size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block Size	512	512	512	1024	1024
Word Size	32	32	32	64	64
Number of Steps	80	64	64	80	80

Note: All sizes are measured in bits.

Message Digest Functions: MD5 and MD6

MD5

Processes an arbitrary-length message and produces a fixed **128-bit** digest. Fast and widely deployed, but **no longer considered collision-resistant** — MD5 is deprecated for security-critical applications.

MD6

Designed to address MD5's weaknesses. Uses a **Merkle tree-like structure** enabling massive parallelism across modern multi-core processors. Submitted to NIST's SHA-3 competition but ultimately not selected as the standard.

- ❏ Neither MD5 nor MD6 should be used for digital signatures or certificate fingerprinting. Prefer SHA-256 or SHA-3 for new designs.

Hashing a File Using OpenSSL

OpenSSL provides built-in support for a wide range of digest algorithms. The commands below demonstrate listing available digest algorithms and computing a SHA-256 hash of a file named `data.txt`.

```
anu@anu-Vostro-5471:~$ openssl dgst -sha256 data.txt
SHA2-256(data.txt)= d0083d8df0fbb4a4bf8ba6c85a8155abd392314fe99b2b61cefe476a46a9af32
anu@anu-Vostro-5471:~$
```

- ❏ The command `openssl dgst -sha256 data.txt` outputs the hex-encoded SHA-256 digest of the file. Even a single character change in the file produces a completely different hash.

```
anu@anu-Vostro-5471:~$ vi data.txt
anu@anu-Vostro-5471:~$ cat data.txt
hello world.
anu@anu-Vostro-5471:~$ openssl list --digest-commands
blake2b512      blake2s256      md5              rmd160
sha1            sha224          sha256           sha3-224
sha3-256        sha3-384        sha3-512         sha384
sha512          sha512-224      sha512-256       shake128
shake256        sm3
anu@anu-Vostro-5471:~$
```

Digital Signature Overview

A digital signature provides **authentication**, **integrity**, and **non-repudiation**.

The sender uses their private key to sign a message hash; the receiver uses the sender's public key to verify it. The two diagrams below show the generic model and the detailed cryptographic steps.

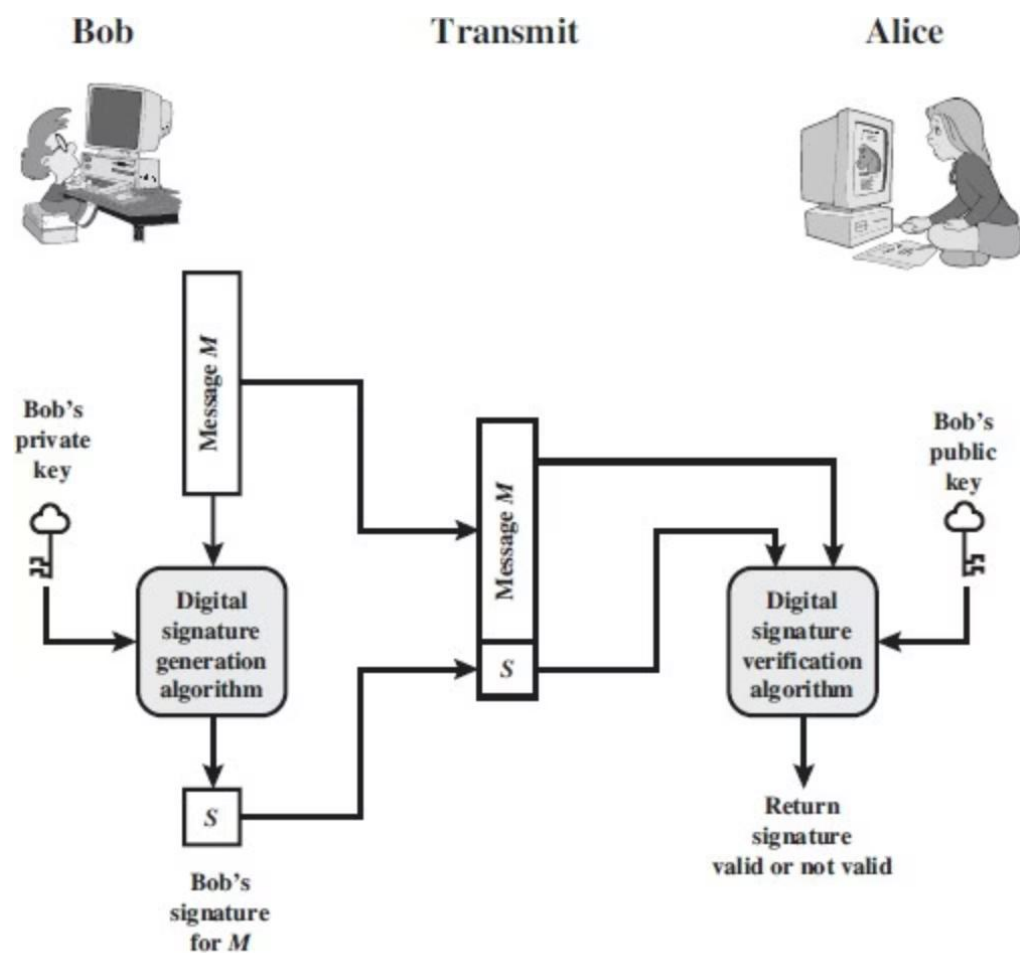


Figure 13.1 Generic Model of Digital Signature Process

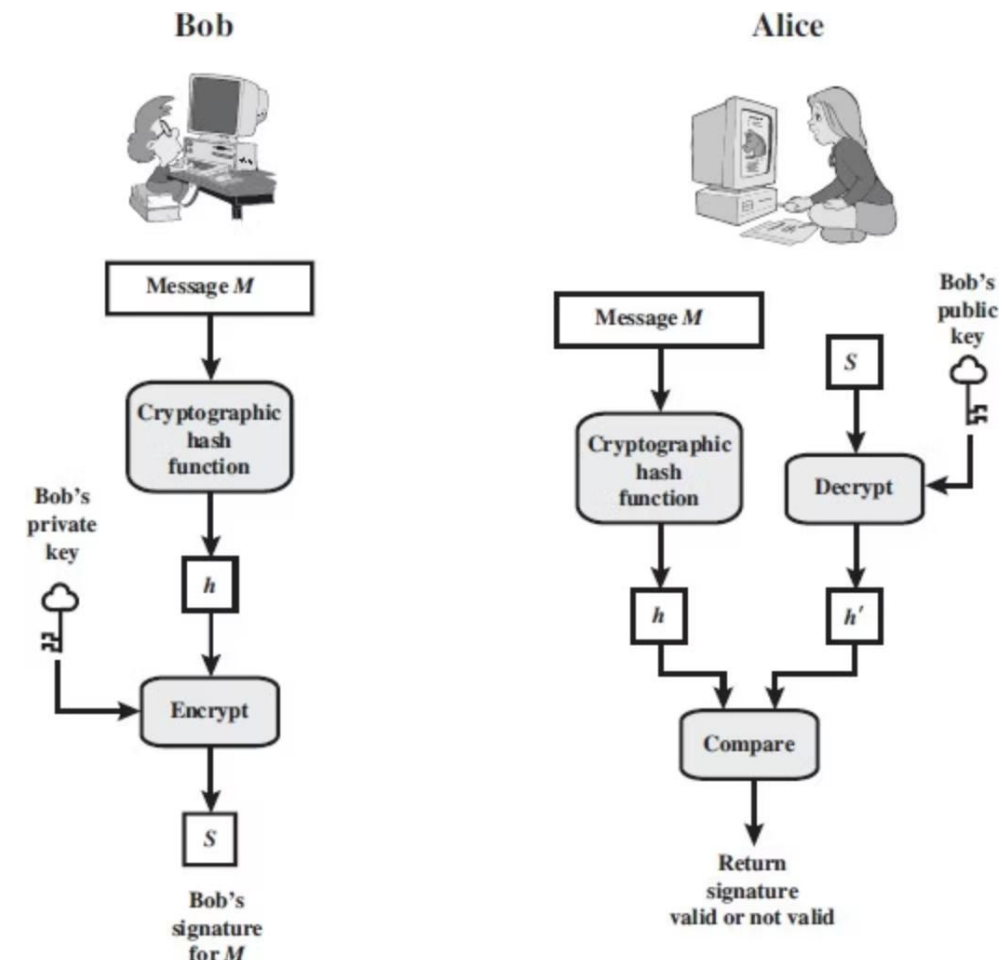


Figure 13.2 Simplified Depiction of Essential Elements of Digital Signature Process

Scenarios for Digital Signature

The scenarios above illustrate why **symmetric message authentication alone is insufficient** — it cannot provide non-repudiation. A properly implemented digital signature must satisfy three core properties:

Identity & Timing

Must verify the author and the exact date and time of signing.

Content Authentication

Must authenticate the contents of the message at the moment of signing.

Third-Party Verifiable

Must be verifiable by an independent third party to resolve disputes.

1. Mary may forge a different message and claim that it came from John. Mary would simply have to create a message and append an authentication code using the key that John and Mary share.
2. John can deny sending the message. Because it is possible for Mary to forge a message, there is no way to prove that John did in fact send the message.

Digital Signature Requirements

For a digital signature scheme to be trustworthy and legally defensible, it must satisfy a strict set of requirements covering both operational ease and cryptographic strength.

→ **Message-Dependent**

The signature must be uniquely bound to the message being signed — the same signer produces different signatures for different messages.

→ **Efficient to Produce & Verify**

Must be computationally practical to generate and to verify for legitimate parties.

→ **Sender-Unique**

Uses information unique to the sender (private key) to prevent both forgery by others and later denial by the sender.

→ **Computationally Infeasible to Forge**

It must be infeasible to create a valid signature for a new message, or to create a fraudulent signature for a given message, without the private key.

Digital Signature Scheme

Two dominant approaches to digital signatures are in use today — RSA and DSA. Each uses a hash of the message but differs in the underlying mathematical operation.

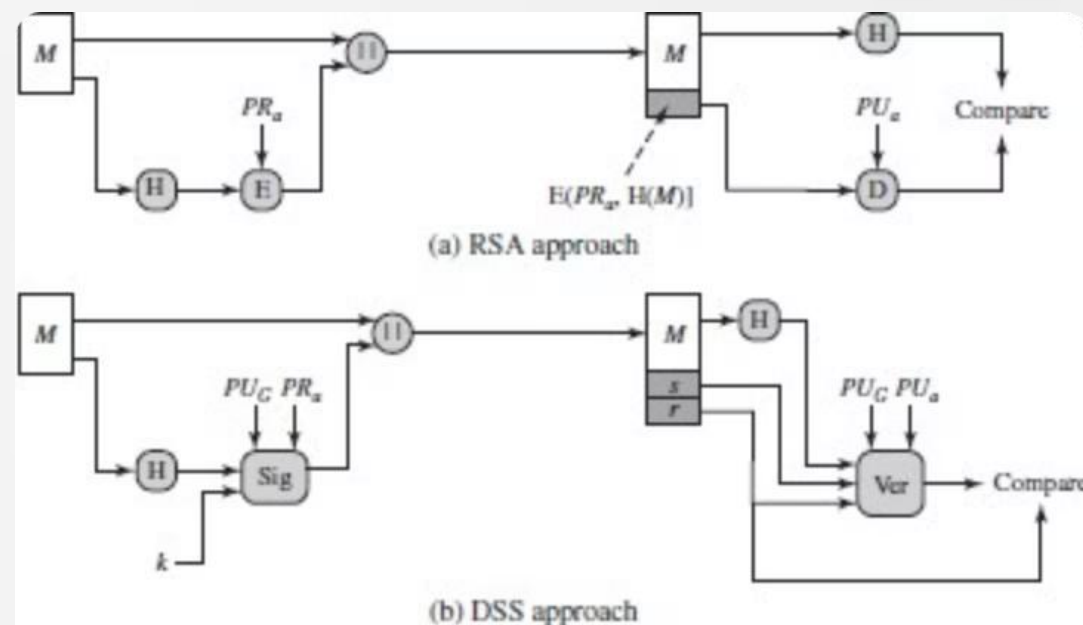


Figure 13.3 Two Approaches to Digital Signatures

RSA Approach

RSA supports encryption, digital signatures, and key exchange. The message hash is encrypted with the sender's private key to produce the signature.

DSA Approach

DSA is a public-key technique used **exclusively** for digital signature generation — it does not support encryption or key exchange. It relies on discrete logarithm hardness.

Digital Signature Standard (DSS)

DSS Overview

The Digital Signature Standard is the U.S. government-approved signature scheme, designed by NIST in the early 1990s and published as **FIPS-186** in 1991. It uses the **SHA hash algorithm** internally. DSS is the standard; **DSA** (Digital Signature Algorithm) is the underlying algorithm.

DSA Properties

- Creates a **320-bit signature**
- Smaller and faster than RSA for signature generation
- Security depends on the computational difficulty of the **discrete logarithm problem**
- Public-key technique — private key signs, public key verifies

Signing a Document Using OpenSSL

The following OpenSSL commands demonstrate generating an RSA key pair and using it to digitally sign a document. First, a 4096-bit RSA private key is generated; then the public key is extracted; finally, the document is signed.

```
anu@anu-Vostro-5471:~$ openssl genrsa -out key3.pem 4096
anu@anu-Vostro-5471:~$ ls
data.txt  file.dec  key3.pem  Public      Templates  test.pem
Desktop   file.enc  key.pem   public1.pem test1.cert  Videos
Documents file.txt  Music     public.pem  test1.pem
Downloads key1.pem  Pictures  snap        test.cert
anu@anu-Vostro-5471:~$ openssl rsa -in key3.pem -pubout > key3.pub
writing RSA key
anu@anu-Vostro-5471:~$ ls
data.txt  file.dec  key3.pem  Pictures  snap  test.cert
Desktop   file.enc  key3.pub  Public    Templates  test.pem
Documents file.txt  key.pem   public1.pem test1.cert  Videos
Downloads key1.pem  Music     public.pem test1.pem
anu@anu-Vostro-5471:~$
```

```
anu@anu-Vostro-5471:~$ openssl pkeyutl -verify -rawin -pubin -inkey key3.pub -in data.txt -sigfile data.sig
Signature Verified Successfully
anu@anu-Vostro-5471:~$
```

```
anu@anu-Vostro-5471:~$ openssl pkeyutl -sign -rawin -inkey key3.pem -in data.txt -out data.sig
anu@anu-Vostro-5471:~$ ls
data.sig  Documents  file.txt  key.pem  public1.pem  test1.cert  Videos
data.txt  Downloads  key1.pem  Music    public.pem   test1.pem
data.zip  file.dec  key3.pem  Pictures  snap         test.cert
Desktop   file.enc  key3.pub  Public    Templates    test.pem
anu@anu-Vostro-5471:~$ cat data.sig
D0A0ej;{0h0000(\0*00C 0000au#]0
      B00bk00U00      00JQ"'S78>000060[Xby0p,7foq0aN-00ZG003I0d0m0800R0*0
      =<0#000000д Ne0c00KJ00'0000r08 00'0-u0070x080hs0?,000T0[0000)Cv0s5000|/0y@'1h
      0?000vsF00000>0q080E"h0h0}#yV00G.000000000j7rU0      0f3\0<r_44;0>0700000a0=Vn000J00000800[0Cp00E00.%00x\00K 0090000E0T00=0! 0000i/00
00x1;000<0000?0"030s.0!Y00{0600000 0S^0000agVvz0P000U
anu@anu-Vostro-5471:~$
```

Symmetric Key Distribution Using Symmetric Keys

Symmetric cryptography requires both parties to share a common secret key before communication. Distributing this key securely — without exposing it to adversaries — is one of the oldest and most critical problems in cryptography.

Direct Physical Delivery

Party A selects a key and physically delivers it to B.
Secure but impractical at scale.

Trusted Third Party

A trusted third party selects and physically delivers the key to both A and B.

Prior Key Encryption

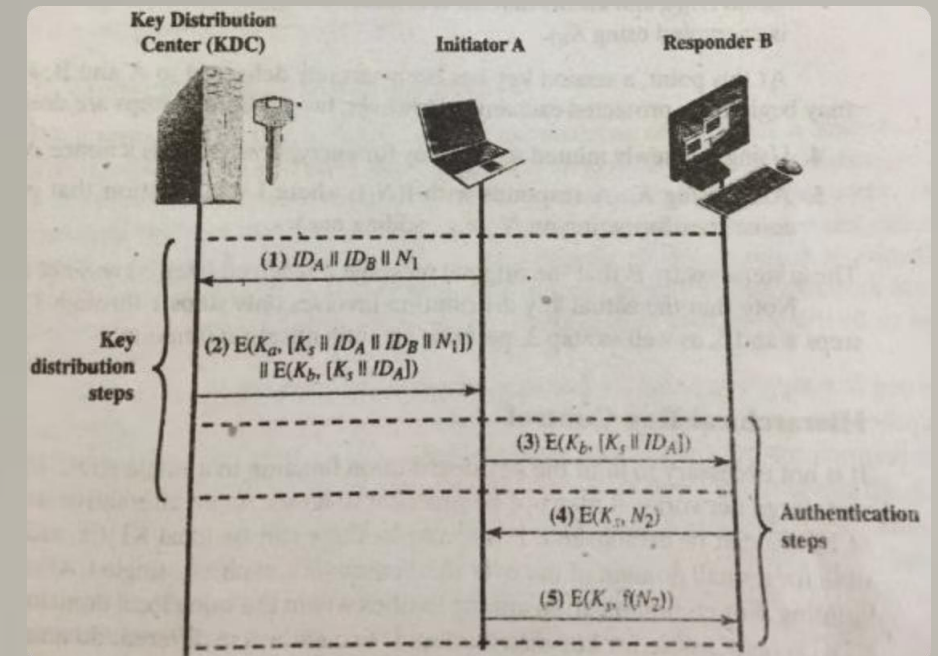
If A and B have communicated before, they can use a previous session key to encrypt a new one.

Relay via Third Party C

If A and B each have a secure channel to a third party C, C can relay a key between A and B.

A Key Distribution Scenario

The diagram below illustrates a typical Key Distribution Center (KDC) scenario, where both parties A and B share master keys with a centralized KDC. The KDC generates and securely delivers a session key to each party, enabling them to communicate without ever exchanging keys directly.



Key Distribution Issues: Hierarchical Key Control

For large-scale networks, a flat KDC model is insufficient. A **hierarchical KDC structure** is established to improve scalability and fault isolation.

Local Domain KDC

Handles key distribution for entities communicating within the same local domain — reduces load on global infrastructure.

Global KDC

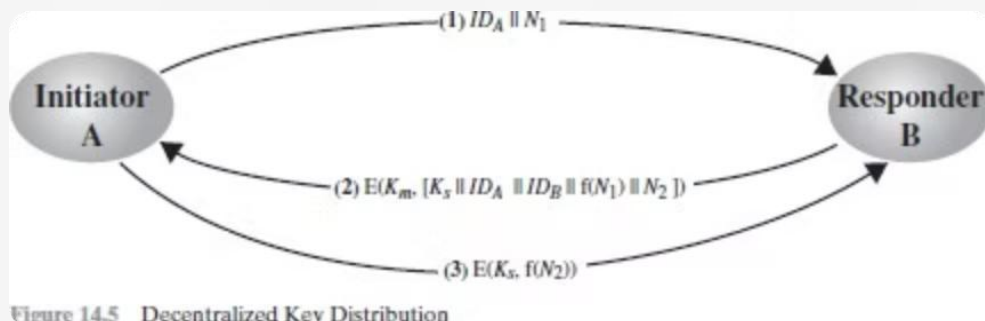
When entities in **different domains** need to communicate, their respective local KDCs communicate through a global KDC to establish a shared key.

Scalable Depth

The number of hierarchy layers depends on network size and geographic scope. Deeper hierarchies support larger, more distributed deployments.

Advantages

Reduces the overhead of master key management and **limits damage** from a compromised KDC to its local area only — containment by design.



Decentralized Key Distribution

A decentralized approach eliminates reliance on a central trusted KDC. Each pair of systems shares a dedicated master key directly, removing a single point of failure or compromise. However, this approach has a significant scalability cost.

- ❏ To support n systems, decentralized key distribution requires $n(n-1)/2$ master keys. For 100 systems, that's 4,950 keys — making management increasingly impractical at scale.

Symmetric Key Distribution Using Asymmetric Encryption

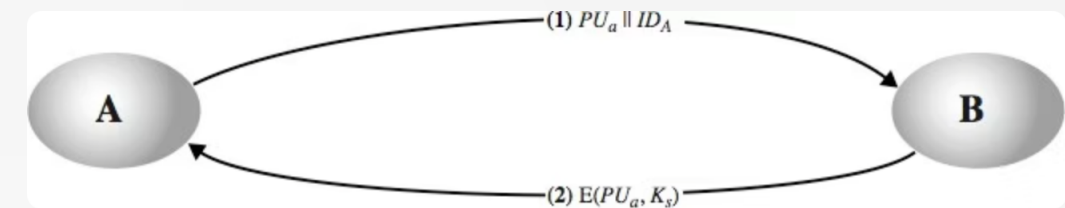
Merkle proposed a simple yet powerful scheme that uses asymmetric (public-key) encryption to securely distribute a symmetric session key. This approach ensures that no shared secret needs to exist before the session begins — and no long-term keys persist after it ends.

No Prior Keys Required

Parties can establish a session key without any pre-shared secret — ideal for first-time communication.

No Keys Persist After

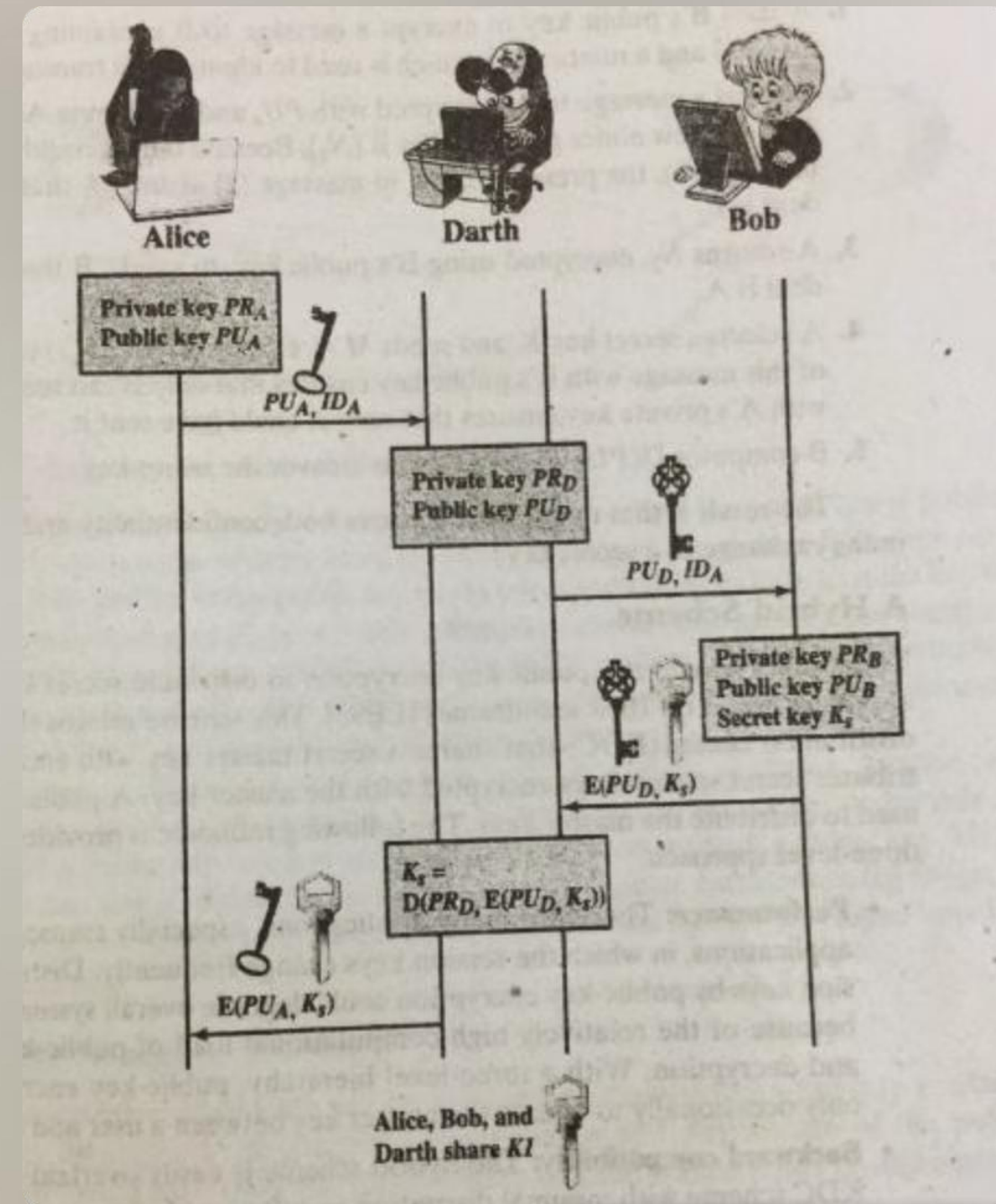
Session keys are ephemeral — once the session ends, the key is discarded, providing forward secrecy.



Man-in-the-Middle Attack

A critical vulnerability of naive public-key exchange is the **Man-in-the-Middle (MitM) attack**. An adversary intercepts both parties' public key exchanges, substitutes their own public key, and can then decrypt, read, re-encrypt, and forward all messages — completely undetected by A and B.

- ❏ MitM attacks motivate the need for authenticated key exchange and Public Key Infrastructure (PKI). Simply exchanging public keys over an untrusted channel is not sufficient.



Secret Key Distribution with Confidentiality and Authentication

To address MitM vulnerabilities, key distribution can be combined with authentication using asymmetric encryption. The protocol below ensures that both confidentiality (only the intended recipient can decrypt) and authentication (each party proves their identity) are achieved simultaneously.

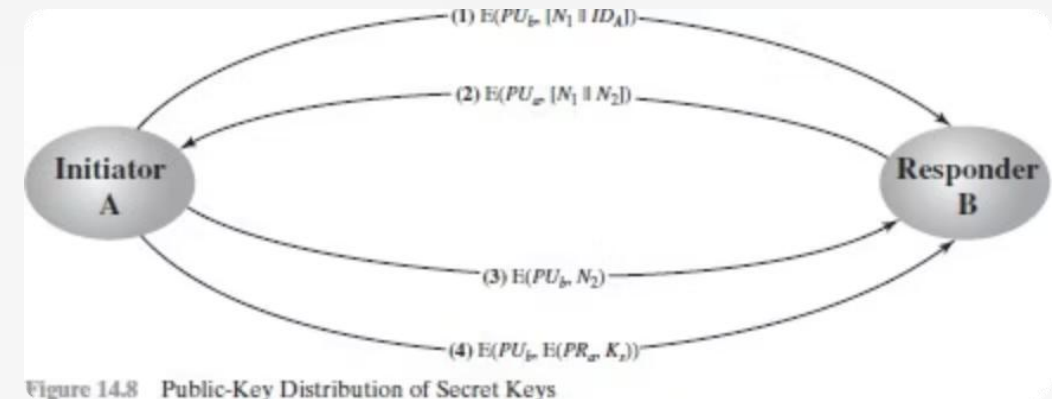


Figure 14.8 Public-Key Distribution of Secret Keys

Distribution of Public Keys

Public key distribution is a fundamental challenge in asymmetric cryptography. Simply broadcasting a public key offers no guarantee it belongs to the claimed owner. Four approaches exist, each with progressively stronger security guarantees.

1

Public Announcement

Users broadcast their own public keys — simple but highly vulnerable to forgery.

2

Public Directory

Keys registered in a trusted directory — more secure but still vulnerable to tampering.

3

Public Key Authority

Centralized authority controls key distribution in real-time — stronger but requires live access.

4

Public Key Certificates

CA-signed certificates provide portable, verifiable proof of key ownership — the gold standard.

Public Announcement

The simplest approach to public key distribution is for users to directly broadcast their public keys to recipients or post them to public forums such as email lists, newsgroups, or social media.

How It Works

A user appends their public key to outgoing email messages, posts it to a public mailing list, or includes it in a directory entry — no infrastructure required.

Major Weakness: Forgery

Any adversary can create a key pair, claim it belongs to someone else, and broadcast it. Until the forgery is discovered, the attacker can impersonate the legitimate user and intercept or forge messages.

- ❑ Public announcement provides **no authentication**. There is no mechanism to verify that a claimed public key truly belongs to the stated party.

Publicly Available Directory

A **trusted public directory** improves upon raw announcements by providing a centralized, managed repository of **{name, public-key}** pairs. Participants register securely, and the directory is periodically published and accessible electronically.

Directory Requirements

Contains verified name–public-key entries.
Participants register with authenticated identity.
Keys can be updated at any time. Directory is published and electronically accessible.

Remaining Vulnerabilities

Still vulnerable to tampering — an attacker who compromises the directory can approve counterfeit public keys or modify existing records to impersonate legitimate entities.

Public Key Authority

A **Public Key Authority (PKA)** tightens control further by acting as a real-time, trusted intermediary for all key lookups. Users must know the PKA's own public key and interact directly with it each time they need another user's key.



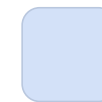
Inherits Directory Properties

Contains all properties of a public directory — verified entries, authenticated registration, and electronic access.



Real-Time Interaction Required

Every key request requires a live interaction with the authority — this can create performance bottlenecks and availability risks.

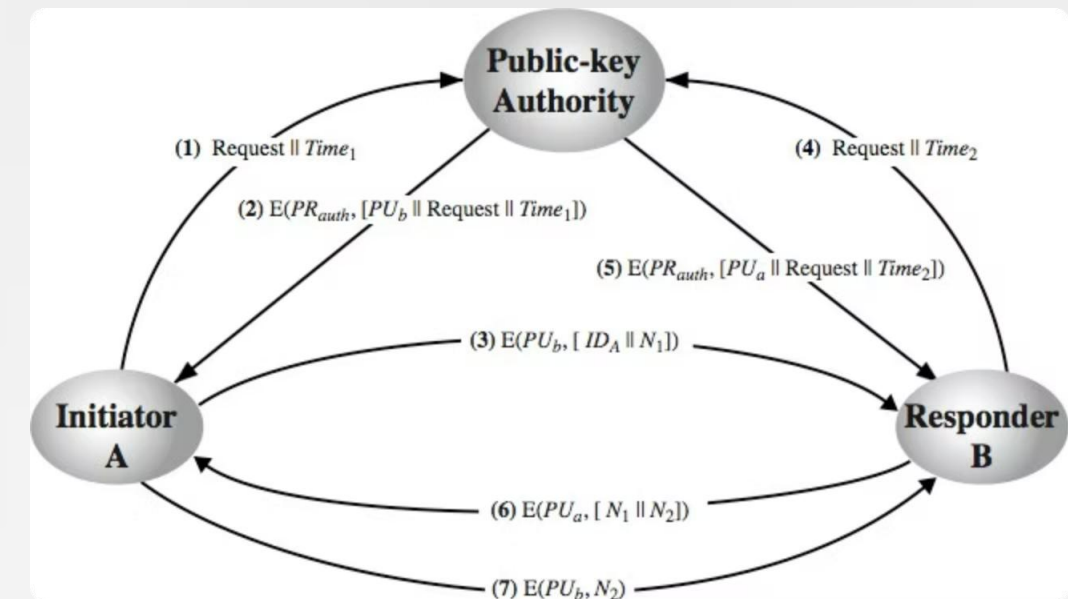


Still Vulnerable to Tampering

Compromise of the authority itself remains a critical risk — a single point of failure for all key distribution in the system.

Public Key Authority — Diagram

The diagram below illustrates how users A and B interact with the Public Key Authority to securely obtain each other's public keys before establishing a protected communication channel.



Public Key Certificates

Public key certificates solve the limitations of centralized directories and real-time authority lookups. A certificate **binds a public key to an identity**, signed by a trusted third party — the Certificate Authority (CA).

Problems with Directory Approach

- Every user must contact the authority in real time to retrieve a key
- The centralized directory is a single point of failure and a target for tampering

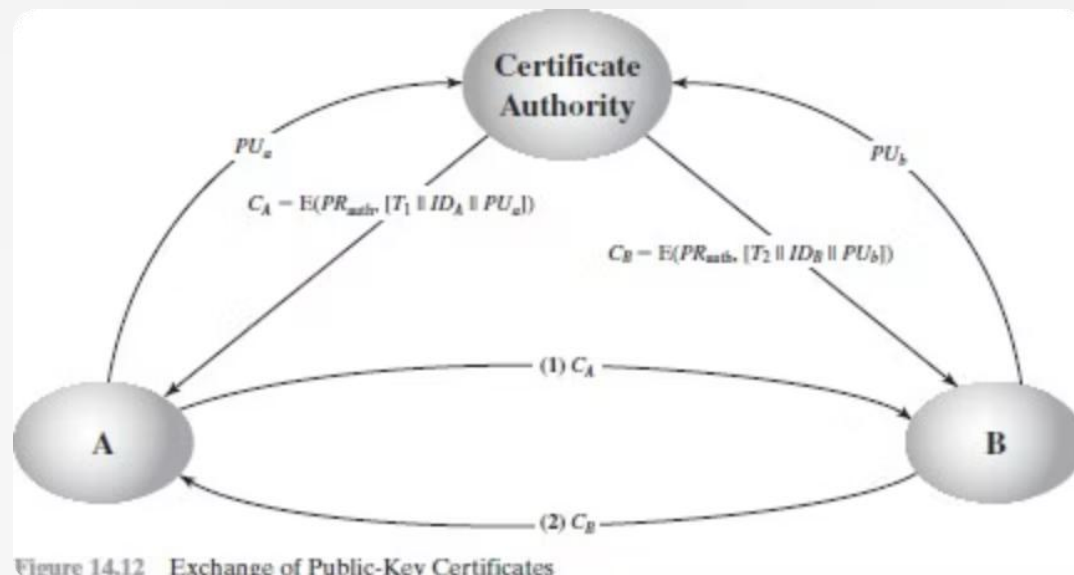
Certificate Solution

A certificate contains the **public key** and the **identity of the key owner**, digitally signed by a trusted CA — such as a government agency or financial institution. Anyone can verify the signature without contacting the CA in real time.

Public Key Certificate — Requirements & Structure

For a public key certificate system to be trustworthy, specific requirements must be met to ensure certificates cannot be forged, misused, or repudiated.

1. Any participant can read a certificate to determine the name and public key of the certificate's owner.
2. Any participant can verify that the certificate originated from the certificate
4. Any participant can verify the currency of the certificate.
3. Only the certificate authority can create and update certificates.



X.509 Certificates

X.509 is the dominant standard for public-key certificates, defined as part of the ITU-T X.500 directory service. It specifies a framework for authentication services and is widely deployed across the internet.

Standard & Specification

Part of ITU-T X.500. Specified in **RFC 5280**. Has gone through 7 versions of progressive enhancement.

Cryptographic Basis

Uses public-key cryptography and digital signatures. Algorithms are not mandated by the standard, but **RSA is recommended**.

Widespread Deployment

Used in **S/MIME** (email security), **IPsec** (network layer), and **SSL/TLS** (web security).



X.509 Certificate Structure

An X.509 certificate contains a structured set of fields that identify the subject, issuer, validity period, and the public key, all bound together by the CA's digital signature.

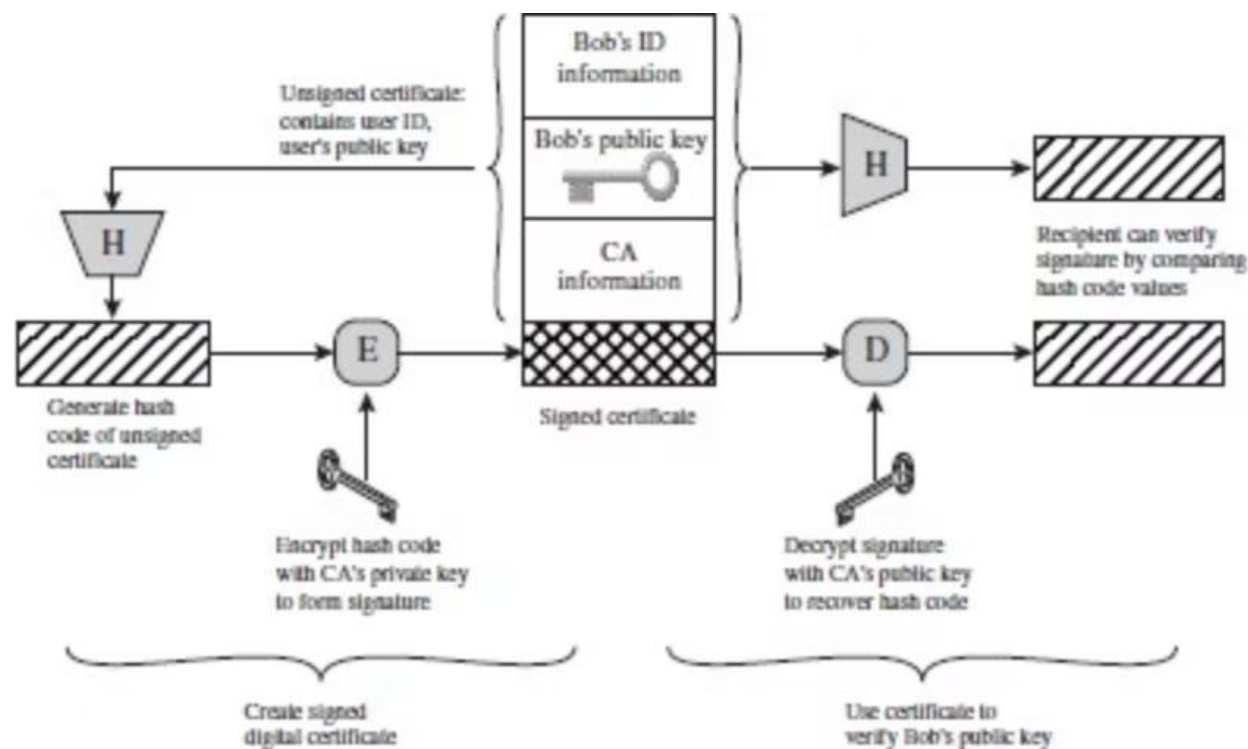
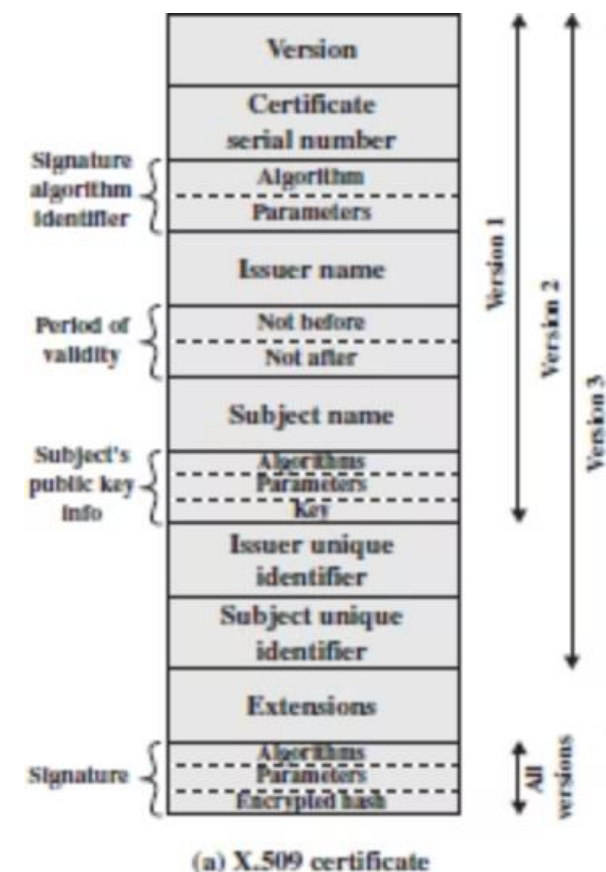


Figure 14.13 Public-Key Certificate Use



Obtaining a User's Certificate

Any user with knowledge of the CA's public key can independently verify that a certificate was issued by that CA — they do not need to contact the CA in real time. Only the CA can create or modify a valid certificate, so certificates can safely be placed in public directories.

❏ **Case Study:** A is certified by CA X_1 and B is certified by CA X_2 . If X_1 and X_2 have exchanged their public keys securely, A can obtain B's certificate through the chain $X_1 \rightarrow X_2 \rightarrow B$, without any prior relationship between A and B.

Step 1 A obtains from the directory the certificate of X_2 signed by X_1 . Because A securely knows X_1 's public key, A can obtain X_2 's public key from its certificate and verify it by means of X_1 's signature on the certificate.

Step 2 A then goes back to the directory and obtains the certificate of B signed by X_2 . Because A now has a trusted copy of X_2 's public key, A can verify the signature and securely obtain B's public key.

A has used a chain of certificates to obtain B's public key. In the notation of X.509, this chain is expressed as

$$X_1 \ll X_2 \gg X_2 \ll B \gg$$

In the same fashion, B can obtain A's public key with the reverse chain:

$$X_2 \ll X_1 \gg X_1 \ll A \gg$$

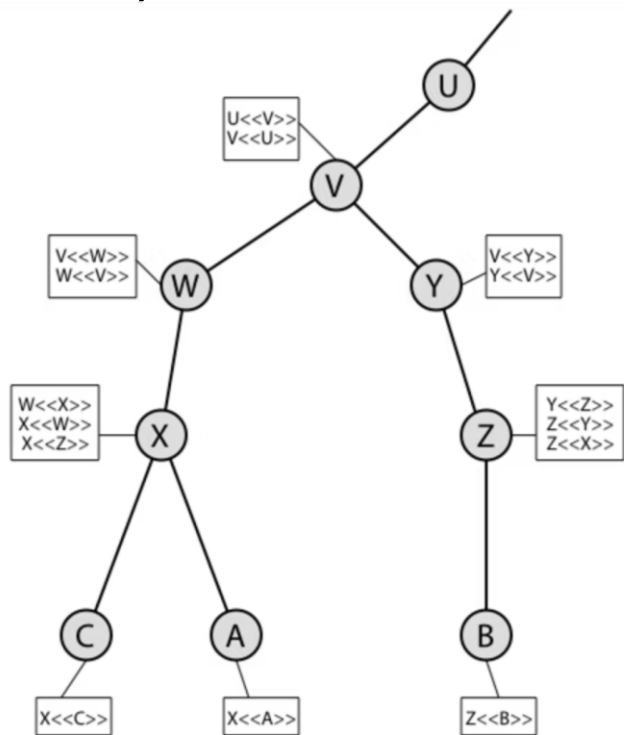
This scheme need not be limited to a chain of two certificates. An arbitrarily long path of CAs can be followed to produce a chain. A chain with N elements would be expressed as

$$X_1 \ll X_2 \gg X_2 \ll X_3 \gg \dots X_N \ll B \gg$$

In this case, each pair of CAs in the chain (X_i, X_{i+1}) must have created certificates for each other.

X.509 Certificate Hierarchy

The X.509 hierarchy defines how CAs are structured and how certificate chains are built. Certificates can be chained across multiple CAs, enabling entities certified by different authorities to still verify each other's credentials.



A to B:

X << W >> W << V >> V << Y >> Y << Z >> Z << B >>

B to A:

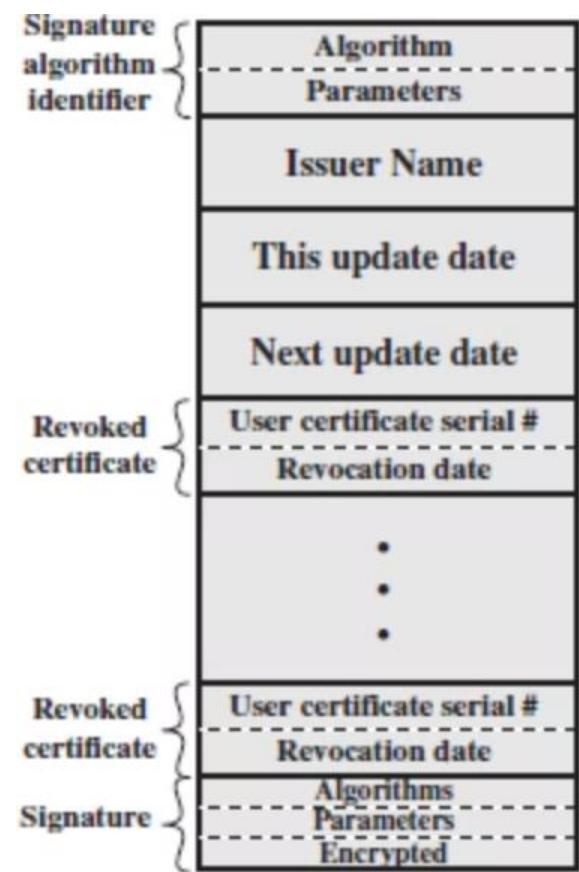
Z << Y >> Y << V >> V << W >> W << X >> X << A >>

Certificate Revocation

Certificates are issued for a defined validity period, but circumstances may require revoking a certificate before it expires. CAs maintain a **Certificate Revocation List (CRL)** — a signed, periodically published list of revoked certificate serial numbers.

Reasons for Revocation

- User's private key has been compromised
- User is no longer certified by this CA
- The CA's own certificate has been compromised



(b) Certificate revocation list

X.509 Version 3 — Extension Framework

Version 3 of X.509 recognized that the fixed fields of earlier versions were insufficient for modern deployment needs. Rather than hard-coding new fields, v3 introduced a **general extension mechanism** allowing arbitrary additional information to be attached to a certificate.

Extension Identifier

An OID (Object Identifier) that uniquely names the extension type — e.g., subject alternative name, key usage, or CRL distribution point.

Criticality Indicator

Marks whether the extension is **critical** (implementations must reject the certificate if they cannot process it) or non-critical (may be safely ignored).

Extension Value

The actual data payload of the extension — e.g., an email address, URL, policy OID, or usage constraint bit mask.

X.509 Version 3 — Certificate Extensions

Three major categories of extensions were introduced in X.509 v3 to support the diverse needs of modern PKI deployment

Key & Policy Information

Conveys information about the subject and issuer keys, including key usage constraints (e.g., signing only, key encipherment) and certificate policy OIDs indicating the assurance level of the certificate.

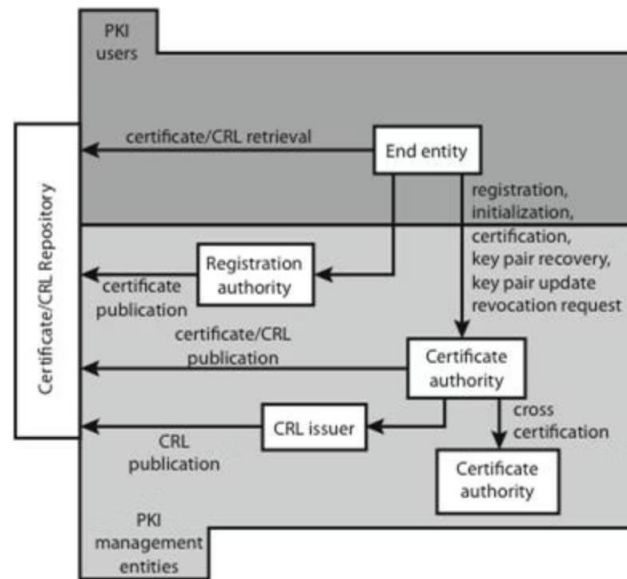
Subject & Issuer Attributes

Supports alternative name formats for the certificate subject and/or issuer — e.g., DNS name, IP address, email address, or URI — beyond the standard Distinguished Name (DN) field.

Certificate Path Constraints

Allows root and intermediate CAs to place constraints on how their certificates may be used by subordinate CAs — limiting the depth of the certificate chain or restricting the name spaces that can be certified.

Public-Key Infrastructure (PKI) — PKIX Architecture



The **IETF PKIX working group** (Public Key Infrastructure X.509) established a formal model based on X.509 for deploying a certificate-based security architecture on the internet. PKIX defines the roles, protocols, and data formats needed to operate a large-scale PKI.



PKIX Management Functions & Protocols

PKIX defines a comprehensive set of management functions that cover the full lifecycle of a certificate — from initial registration through revocation. These are supported by standardized protocols.

Management Functions

- Registration — entity enrolls with the PKI
- Initialization — entity sets up cryptographic material
- Certification — CA issues a certificate
- Key pair recovery — restore lost keys
- Key pair update — replace expiring keys
- Revocation request — invalidate a certificate
- Cross certification — CAs mutually recognize each other

Supporting Protocols

CMP (Certificate Management Protocol) — a comprehensive protocol for managing the full certificate lifecycle between PKI entities.

CMC (Certificate Management over CMS) — a simpler protocol built on top of Cryptographic Message Syntax, often used for certificate enrollment requests.

Types of Certificates: EV and OV

Not all certificates offer the same level of assurance. CAs issue different certificate types based on the rigor of the identity verification process performed before issuance.

Extended Validation (EV) Certificates

Require the most rigorous verification: legal and physical existence of the organization, cross-checking government records, and confirming exclusive domain rights. The browser URL bar turns **green** for EV-secured sites. These are the most expensive certificates and may take several days to issue.

Organization Validation (OV) Certificates

Involve moderate identity checks — the CA verifies the organization's existence and the domain name, typically followed by a phone call from the CA to confirm the applicant. Less rigorous than EV but still provides meaningful assurance about the organization.

Types of Certificates: Domain Validation (DV)

Domain Validation certificates represent the lowest tier of certificate assurance. They verify only that the applicant controls the domain — not the identity of the organization behind it.

Verification Process

The CA confirms the applicant's right to use a specific domain name by comparing it against existing domain name databases and checking internet directories — no organization identity is verified.

Speed & Cost

DV certificates are the **least expensive** type and are typically issued within **minutes** of application — making them popular for small websites, blogs, and development environments.

Limitations

DV certificates only guarantee encrypted communication to the domain — they provide **no assurance** about who operates the website. Phishing sites can and do use DV certificates.

CA Hierarchy

Certificate Authorities operate through a strict **tree-like hierarchical structure** where trust flows only downward — from root CAs at the top, through intermediate CAs, down to end-entity certificates.

1

End-Entity Certificates

Issued to websites, users, or devices. Verified by intermediate CA signatures.

2

Intermediate-Level CAs

CAs below the root that issue end-entity certificates. Their certificates are signed by root or higher-level intermediate CAs.

3

Root CAs

Self-signed. Pre-loaded in operating systems and browsers. Examples: Comodo, DigiCert, GoDaddy, IdenTrust. Found in `/etc/ssl/certs/` on Linux.


```
anu@anu-Vostro-5471:/etc/ssl/certs$ cat vTrus_Root_CA.pem
-----BEGIN CERTIFICATE-----
MIIFVjCCAz6gAwIBAgIUQ+NxE9izWRRdt86M/TX9b7wFjUuWdQYJKoZIhvcNAQEL
BQAwQzELMAkGA1UEBhMCQ04xHDAaBgNVBAoTE2lUcnVzQ2hpbmEgQ28uLEx0ZC4x
FjAUBgNVBAMTDXZUcnVzIFJvb3QgQ0EwHhcNMTgwNzMxMDcyNDA1WhcNNDMwNzMx
MDcyNDA1WjBDMQswCQYDVQQGEwJDTjEjEcMBoGA1UEChMTaVRydXNDaGlueSBDbY4s
THRkLjEwMBQGA1UEAxMNdlRydXMgUm9vdCBDQTCCAiIwDQYJKoZIhvcNAQEBBQAD
ggIPADCCAggCggIBAL1VfGHTuB0EYgWgrmy3cLRB6ksDXhA/kFocizuwZotsSKYc
IrrVQJLuM7IjWcm0vFjai57QGfIvWcaMY1q6n6MLsLOaXLoRuBLpDLvPbmyAhykU
AyyNJJrIZIO1aqwTLDpxn9wsYTwaP3BVm60AUUn/PBLn+NvqcwBauYv6WTEN+VRS+
GrPSbcKvdmaVayqwlHeFXgQPYh1jdfdr58tbmnDsPmcF8P4HCIDPKNsFxbQnL4Z9
8Cfe/+Z+M0jnCx5Y0ScrUw5XSmXX+6KAYPxMvDVTAWqXcoKv8R1w6Jz1717CbMdH
flqUhSZNO7rrT0iwcJlwp2dCZtOtZcFrPUGoPc2BX70kLJrxLT5Z0rpGgrIDajt
J8nU5705q4IikCc9Kuh8k0+8T/3iCiSn3mUkpF3qwhYw03dQ+A0Em5Q2AXPKBlm
0zvc+gRGE1WKyURHuFE5Gi7oNOJ5y1lKcN+8pu8fA2dqWsslypPZUxlmPCdiKYZN
pGvu/9ROutW04o5IWgAZCfEF2c6Rsffr6TlP9m8EQ5pV9T4FFL2/s1m02I4zhKOQ
UqqzApVg+QxMaPnu1RcN+HFxtSXkKe5lXa/R7jwXC1pDxaWG6iSe4gUH3DRCEpHW
OXSuTEGC2/KmSNGzm/Mzqv0mwMVO9fSddmPmAsYiS8GVP1BkLFTltvA8Kc9XAgMB
AAGjQjBAMB0GA1UdDgQWBRRUYnBj8XWEQ1i00RYgscasGrz2iTAPBgNVHRMBAf8E
BTADAQH/MA4GA1UdDwEB/wQEAwIBBjANBgkqhkiG9w0BAQsFAAOCAGeAKbqSSaet
8PFww+SX8J+pJdVrnjT+5hpk9jprUrIQeBqfTNqK2uwcN1LgQkv7bHbKJAs5EhWd
nxEt/Hlk3ODg9d3gV8mlsnZwUKT+twpw1aA08XXXtUm6EdGz20yC/+s0xL9kLX1j
bhd47F18imjrjld22Vke+rxSH0Ws8HqA70xvdq6R2xCOBNyS36D25q5J08FsEhvM
Kar5CKXiNxTKsbhm7xqC5PD48acWabfbqWE8n/Uxy+QARsIvdLGx14HuqCaVvIiv
TDUHKgLKeBRtRytAVunLkmChZwOgzoy8sHJnxDH02zTlJQNgJXtxm0TAGytfDELs
S8VZCAeHvsXDf+eW2eHcKJfWjwXj9Zt0yh1QRwVTsMo554WgicEF0wE30z9J4nfr
I8iIZjs90XYhRvHsXy0466JmdXTBQPfYaJqT4i2pLr0cox7IdMakLXogqzu4sEb9
b91fUlV1YvCXoHzXOP0l382gmxDpi7g4Xl7FtKYCNqEeXxzP4padKar9mK5S4fNB
UvupLnKWnyfjqnN9+BojZns7q2WwMgFLFT49ok8MKzWixtlnEjUwzXYuFrOZnk1P
Ti07NEPhmg4NpGaXutIcSkwsKouLgU9xGqndXht7CMUADTdA43x7VF8vhV929ven
sBxXVsFy6K2ir40zSbofitzmdHxghm+Hl3s=
-----END CERTIFICATE-----
```

Why Intermediate-Level CAs Are Necessary

Root CA private keys are among the most sensitive secrets in all of digital security infrastructure. The use of intermediate CAs insulates root CAs from day-to-day certificate operations — and from the consequences of key compromise.

Root Key Compromise: Catastrophic

Root CA public keys are pre-loaded into every OS and browser. A compromised root key requires a software update pushed to **all devices** — impractical and potentially impossible for embedded systems.

Intermediate Key Compromise: Manageable

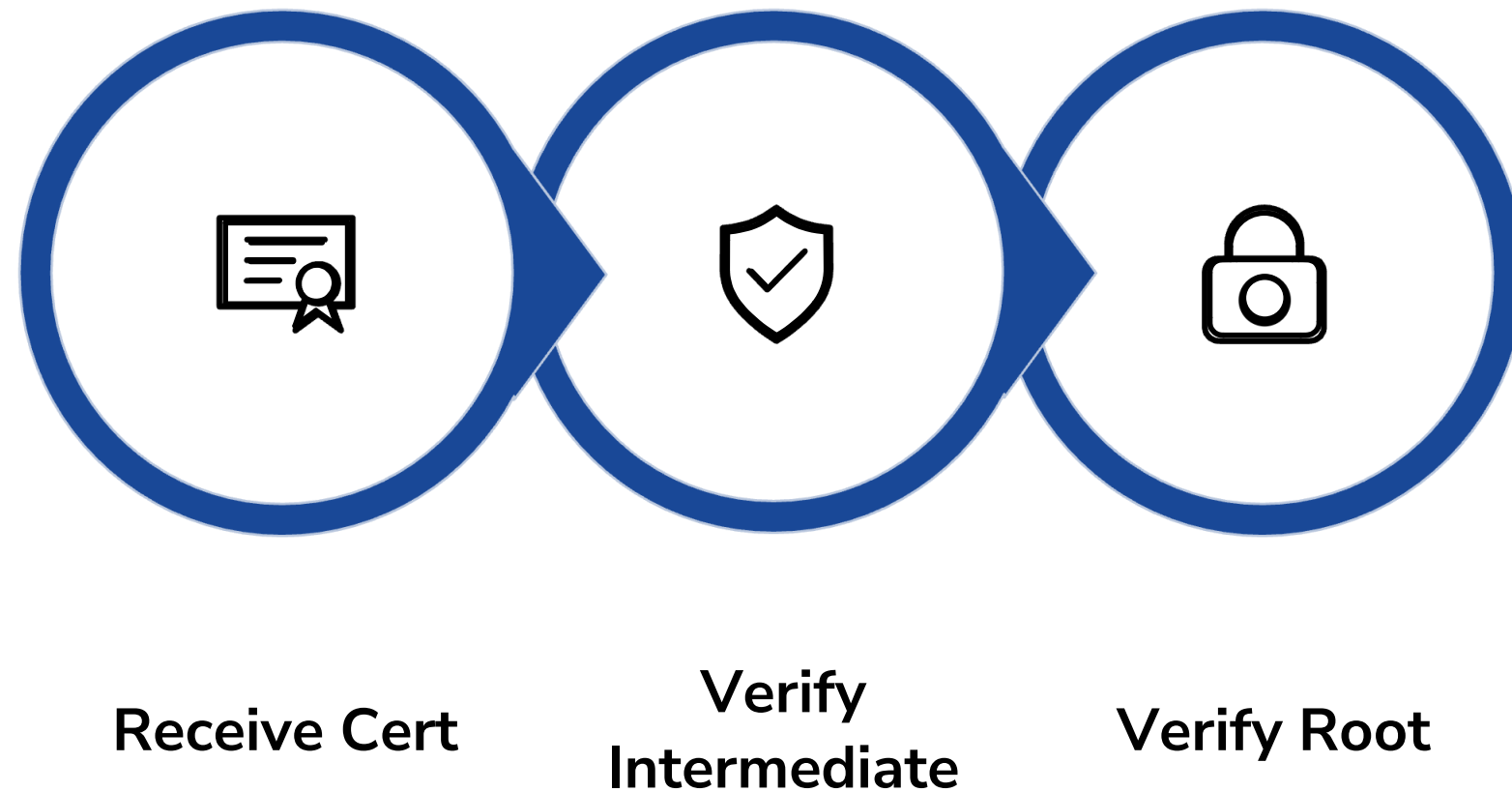
If an intermediate CA's private key is compromised, the affected certificates are added to a **Certificate Revocation List (CRL)** maintained by the parent CA — no OS-level update is required.

Recovery Path

The compromised intermediate CA can have its certificate revoked and then proceed to **re-issue fresh certificates** to all affected parties under a new key pair, with minimal disruption.

Certificate Validation — Chain of Trust

When your browser receives a certificate, it must verify that the certificate is signed by a trusted CA. This process is **recursive** — the browser walks up the certificate chain, verifying each signature, until it reaches a pre-installed root certificate.



To save repeated round-trips, web servers often send the **entire certificate bundle** (the end-entity certificate plus all intermediate CA certificates) so the browser can validate the entire chain locally. A CA authenticates: (1) the certificate itself, (2) the certificate's owner, and (3) the owner's public key.

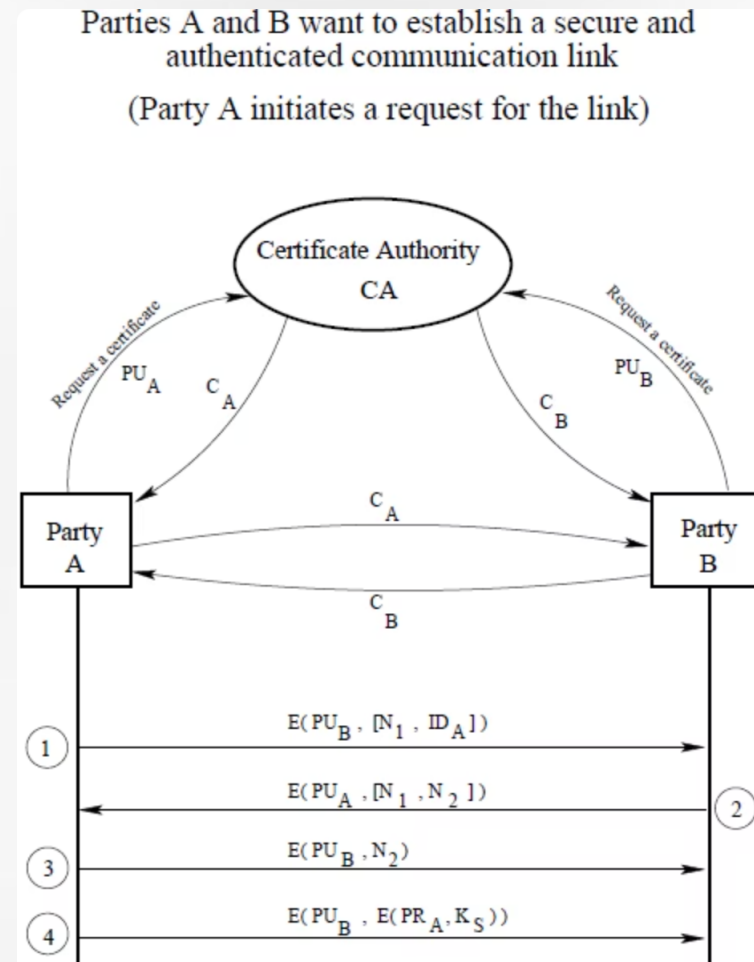


Figure 1: Messages exchanged between two parties for acquiring each other's CA authenticated public keys. (This figure is from Lecture 13 of "Computer and Network Security" by Avi Kak.)

Figure Description

This diagram illustrates the process of two parties (A and B) establishing a secure and authenticated communication link using a Certificate Authority (CA).

Setup Phase (Top of Diagram)

- Both **Party A** and **Party B** independently contact the **Certificate Authority (CA)**:
 - Each requests a certificate from the CA
 - Each sends their public key (PU_A and PU_B) to the CA
 - The CA issues signed certificates C_A and C_B back to each party

Certificate Exchange Phase (Middle)

- Party A sends its certificate C_A directly to Party B
- Party B sends its certificate C_B directly to Party A
- Each party can now extract and verify the other's **authenticated public key**

Key Exchange Protocol (Steps 1-4, Bottom)

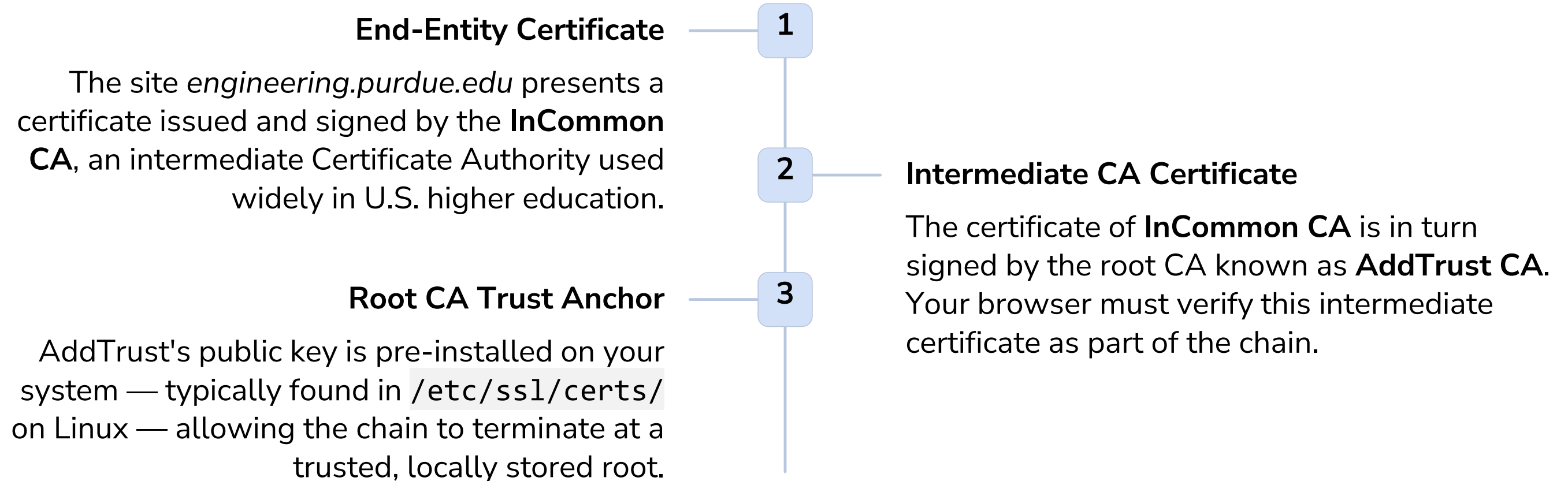
1 $E(PU_B, [N_1, ID_A])$ A sends a nonce N_1 and its identity, encrypted with B's public key
 2 $E(PU_A, [N_1, N_2])$ B responds with N_1 (proving identity) + new nonce N_2 , encrypted with A's public key
 3 $E(PU_B, N_2)$ A returns N_2 to B, confirming mutual authentication
 4 $E(PU_B, E(PR_A, K_S))$ A sends the session key K_S , signed with A's private key and encrypted with B's public key

Purpose

This protocol achieves mutual authentication and secure session key distribution — both parties verify each other's identity via CA-issued certificates before exchanging a shared symmetric session key.

Case Study: Purdue Engineering Website

This case study walks through a real-world certificate chain to illustrate how certificate validation works in practice when a browser connects to a university web server.



Case Study — Certificate Screenshots

The screenshots below show what a real browser certificate inspection looks like — including the certificate hierarchy, validity period, and the chain from end-entity to intermediate to root CA.

```
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      7f:71:c1:d3:a2:26:b0:d2:b1:13:f3:e6:81:67:64:3e
    Signature Algorithm: sha1WithRSAEncryption
    Issuer: C=SE, O=AddTrust AB, OU=AddTrust External TTP Network, CN=AddTrust External CA Root
    Validity
      Not Before: Dec 7 00:00:00 2010 GMT
      Not After : May 30 10:48:38 2020 GMT
    Subject: C=US, O=Internet2, OU=InCommon, CN=InCommon Server CA
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      RSA Public Key: (2048 bit)
        Modulus (2048 bit):
          00:97:7c:c7:c8:fe:b3:e9:20:6a:a3:a4:4f:8e:8e:
          34:56:06:b3:7a:6c:aa:10:9b:48:61:2b:36:90:69:
          e3:34:0a:47:a7:bb:7b:de:aa:6a:fb:eb:82:95:8f:
          ca:1d:7f:af:75:a6:a8:4c:da:20:67:61:1a:0d:86:
          c1:ca:c1:87:af:ac:4e:e4:de:62:1b:2f:9d:b1:98:
          af:c6:01:fb:17:70:db:ac:14:59:ec:6f:3f:33:7f:
          a6:98:0b:e4:e2:38:af:f5:7f:85:6d:0e:74:04:9d:
          f6:27:86:c7:9b:8f:e7:71:2a:08:f4:03:02:40:63:
          24:7d:40:57:8f:54:e0:54:7e:b6:13:48:61:f1:de:
          ce:0e:bd:b6:fa:4d:98:b2:d9:0d:8d:79:a6:e0:aa:
          cd:0c:91:9a:a5:df:ab:73:bb:ca:14:78:5c:47:29:
          a1:ca:c5:ba:9f:c7:da:60:f7:ff:e7:7f:f2:d9:da:
          a1:2d:0f:49:16:a7:d3:00:92:cf:8a:47:d9:4d:f8:
          d5:95:66:d3:74:f9:80:63:00:4f:4c:84:16:1f:b3:
          f5:24:1f:a1:4e:de:e8:95:d6:b2:0b:09:8b:2c:6b:
          c7:5c:2f:8c:63:c9:99:cb:52:b1:62:7b:73:01:62:
          7f:63:6c:d8:68:a0:ee:6a:a8:8d:1f:29:f3:d0:18:
          ac:ad
        Exponent: 65537 (0x10001)
  X509v3 extensions:
    X509v3 Authority Key Identifier:
      keyid:AD:BD:98:7A:34:B4:26:F7:FA:C4:26:54:EF:03:BD:E0:24:CB:54:1A

    X509v3 Subject Key Identifier:
      48:4F:5A:FA:2F:4A:9A:5E:E0:50:F3:6B:7B:55:A5:DE:F5:BE:34:5D
    X509v3 Key Usage: critical
    X509v3 Certificate Sign, CRL Sign
    X509v3 Basic Constraints: critical
      CA:TRUE, pathlen:0
    X509v3 Certificate Policies:
      Policy: X509v3 Any Policy

    X509v3 CRL Distribution Points:
      URI:http://crl.usertrust.com/AddTrustExternalCARoot.crl

    Authority Information Access:
      CA Issuers - URI:http://crt.usertrust.com/AddTrustExternalCARoot.p7c
      CA Issuers - URI:http://crt.usertrust.com/AddTrustUTNSGCCA.crt
      OCSP - URI:http://ocsp.usertrust.com

    Signature Algorithm: sha1WithRSAEncryption
      93:66:21:80:74:45:85:4b:c2:ab:ce:32:b0:29:fe:dd:df:d6:
      24:5b:bf:03:6a:6f:50:3e:0e:1b:b3:0d:88:a3:5b:ee:c4:a4:
      12:3b:56:ef:06:7f:cf:7f:21:96:56:3b:41:31:fe:e1:aa:93:
      d2:95:f3:95:0d:3c:47:ab:ca:5c:26:ad:3e:f1:f9:8c:34:6e:
      11:be:f4:67:e3:02:49:f9:a6:7c:7b:64:25:dd:17:46:f2:50:
      e3:e3:0a:21:3a:49:24:cd:c6:84:65:68:67:68:b0:45:2d:47:
      99:cd:9c:ab:86:29:11:72:dc:d6:9c:36:43:74:f3:d4:97:9e:
      56:a0:fe:5f:40:58:d2:d5:d7:7e:7c:c5:8e:1a:b2:04:5c:92:
      66:0e:85:ad:2e:06:ce:c8:a3:d8:eb:14:27:91:de:cf:17:30:
      81:53:b6:66:12:ad:37:e4:f5:ef:96:5c:20:0e:36:e9:ac:62:
      7d:19:81:8a:f5:90:61:a6:49:ab:ce:3c:df:e6:ca:64:ee:82:
      66:39:45:95:16:ba:41:06:00:98:ba:0c:56:61:e4:c6:c6:86:
      01:cf:66:a9:22:29:02:d6:3d:cf:c4:2a:8d:99:de:fb:09:14:
      9e:0e:d1:d5:c6:d7:81:dd:ad:24:ab:ac:07:05:e2:1d:68:c3:
      70:66:5f:d3
    -----BEGIN CERTIFICATE-----
    MIIEvzCCA6ugAwIBAgIQf3HB06ImnNKxE/PagWdkPjANBgkqhkiG9w0BAQUFADBv
    MQswCQYDVQQGEwJTRTEUMBIGA1UEChMLQWRkLWVhJ1c3QGU1xjAkBgNVBAAsTHU1fXk
    ZFRydXNOIEV4dGVybnFzIFRlUUCBOGZlR3b3JrMSIwIAYDVQQDEs1BZZGRUcnVzdCBF
    eHRlcm5hbCBQZSStb290bG4KDTcwMTIwNzAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAw
    UTELMARkA1UEBhMCVWMeEjAQBgNVBAoTCU1udGVybnVOMjEwMDAwMDAwMDAwMDAwMDAw
    b21tb24xGzAZBgNVBAMTEkludGVybnVOMjEwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAw
    AQEBBQADggEPADCCAQoCggEBAJd8x8j+s+kgaqUkT46UNFYGa3peqbCbSGErNp8p
    4zQKR6e7e96qavvrgpWPh1/r3WaqEznIGdhGg2GwcrBh6+sTuTeYhsnbnGyr8YB
    +xdv26wUWexvPzN/ppgLEOI4r/V/hW00dASd9ieGx5uP53EqCPQDAkBJJH1AV49U
    4FR+thNIYfHezg68tvpMnLLZDY15puCqzQyRnqXfq307yhR4XEcpcrFup/R2mD3
    /+d/8tnaoSOPSRan0wCSz4pH2U341ZVn03T5gGMAT0yEFh+zs9SQfoU7e6JXWagsJ
    iyxr1vvpJPJactSsWJ7cwFif2Na2Gig7nqoJR8p89AYrKOCaVAAaDCAJcwggFz
    MB8GA1UdIwQYMBaAFK29aHo0cCb3+sQwV08DwaAkylQaMBOGA1UdDgQWBBr1Tr6
    LOqaXu8Q82t7VaXe9b40XTA0BgNVHQ8BAf8EBAMCAQYwEgYUVVROTATQh/BagwBgEB
    /wIBADARBgNVHSAECjA1MAYGBFUDIAAARAYDVROfBD0wOzA50DegNYyzaHR0cDov
    L2NyYbC51c2VydHJ1c3QyY29tL0FkZFRydXNOIEV4dGVybnVOMjEwMDAwMDAwMDAw
    BggrBgEFBQcBAQSBpjCBzaA/BggrBgEFBQcwoA1Zn03T5gGMAT0yEFh+zs9SQfoU7e6JXWagsJ
    c3QyY29tL0FkZFRydXNOIEV4dGVybnVOMjEwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAw
    dHRwOisvY3J0LnVzZXJ0cnVzdC5jb20vQWRkLWVhJ1c3QGU1xjAkBgNVBAAsTHU1fXk
    KwYBBQUHMAKGGBw0dHAGLy9vY3NwLnVzZXJ0cnVzdC5jb20vQGU1xjAkBgNVBAQEF
    BQADggEBANnIYBORYVlwqyUmrAp/t3f1iRbvNqb1A+DhuzDYiJw+7EpBITVn8G
    f89/LZVW00Ex/uGqkKVB8UNPEerylwmrT7s+Yw0bhG+9GfjAkn5pmx7ZCXdf0by
    UUPjCiE6SSYDxoRlaGdosEUtR5nNnKqKRFy3WacNkN089Sxnlag/19AWNVLV1368
    xY4asgRcknY0ha0uBs7Io9jrFCeR3s8XMIFFttnYsrTfk9e+WXCADNumsYn0ZgYr1
    kGQnSavOPN/nymTugnU6RZUWukEGAj:6DFZ5MbGhgHPZqkiKQLWPC/EXo2Z3vsJ
    FJ400dXG14HdR5SrrAcF4h1ow3BaX9M=
    -----END CERTIFICATE-----
```




Do It Yourself — Hands-On Exercise

Apply what you have learned by inspecting real X.509 certificate chains in your browser. This exercise bridges theory and practice — you will see first-hand how certificate hierarchies, validity periods, and extensions work in live deployments.

01

Open Chrome & Firefox

Launch both Google Chrome and Mozilla Firefox on your machine.

02

Visit Any HTTPS Website

Navigate to any `https://` website — for example, *engineering.purdue.edu* or your university portal.

03

View the Certificate

Click the padlock icon in the address bar → Certificate (or Certificate Information). Explore the chain: end-entity → intermediate CA → root CA.

04

Compare Browsers

Note any differences in how Chrome and Firefox display the certificate hierarchy, extensions, and validation status.

Creating Your Own Certificate with OpenSSL

You can generate your own self-signed certificate using OpenSSL — useful for local testing and understanding the certificate structure. The commands below demonstrate generating a private key, creating a certificate signing request (CSR), and self-signing the certificate.

```
anu@anu-Vostro-5471:~$ openssl req -new -newkey rsa:1024 -days 365 -nodes -x509 -keyout test1.pem -out test1.cert
.....+.....
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:61
State or Province Name (full name) [Some-State]:nsw
Locality Name (eg, city) []:sydney
Organization Name (eg, company) [Internet Widgits Pty Ltd]:temp com
Organizational Unit Name (eg, section) []:ict
Common Name (e.g. server FQDN or YOUR name) []:.
Email Address []:..
-----
```

```
anu@anu-Vostro-5471:~$ ls
Desktop  Downloads  file.enc  key1.pem  Music      Public      public.pem  Templates  test1.pem  test.pem
Documents  file.dec  file.txt  key.pem   Pictures   public1.pem  snap        test1.cert  test.cert  Videos
anu@anu-Vostro-5471:~$ cat test1.pem
-----BEGIN PRIVATE KEY-----
MIICeQIBADANBgkqhkiG9w0BAQEFAASCAmMwgGJfAgEAAoGBAKFQf/LA4Rz20wwt
41NwkpQYEznfIcAx+0jhoyqLY233tdrYDQpRyqAqY6H/WgJUA/R8PPNiXX2hmE+x
2IVLR3g9ge/fo4KUWACA5QSu3kBy3XSbzfXDLv69Sbylk/UVRxK/+YcJ/SvzDYz
iZUWIxLFRG0m/G/LVsRZR/Sg/35AgMBAAECgYEAnYqEqot8TLCbmjX0gTq7rC4m
+KnVyGIHyxGxzIBhLpBw5h2B/sYKYmEbD1SpjRu+GItFHut8pfSrGI/12cL99cK
qPwtDMLXt4MEsGzIUo9ky6p8LSsjEgO7SHDFAP7Zv2auCBSfIvwRgLIJMVjmkzCr
0E8A1co6fnA/FyipBkECQQDMPw60nC8Ff/acH588pnsfFVc34qWvWc4T/yfjPBtX
pdWLSNEzzjxCwiUVC2Eh4gQ41GcAwHDEstkErgCwz4lAkEayjCPIjLZlQBpJAfD
KZRT4SbExlRLYWTkhOU9WxoZVM+nw3aB/RywsY+gNNWgJgn4R1dsa+3jksPkc0n+
DyPmRQJBAKf04zEm5Uc6hI5vOLBnA99c9ETZ0pPASTtpEbBri6BGAVZ7dtZ+imo7
BYyr+OP7SgK9ca6hldAFYBA/h0nYKfUCQQChZSYPCQvSLu0+yRt2o1pZjbLhhhjv
J7Rr2jwq7qhuVJSCKIsW1ZHFCxsDUBG58Cdp+1UmylwwmYJQrS3KNCPpAkEAo9LC
gAW6wgffFyy9LS2eBel+uUBQoKNDCL9bz+1HJJTLmjwsyTBUD5f4xRnijKJj7L7/
XQN9hgU/m61T4+fsdQ==
-----END PRIVATE KEY-----
```

```
anu@anu-Vostro-5471:~$ cat test1.cert
-----BEGIN CERTIFICATE-----
MIICdjCCAd+gAwIBAgIUJiYkm1SPLNr4zsevrB5nrA+20aQwDQYJKoZIhvcNAQEL
BQAwTTElMAkGA1UEBhMCNjExDDAKBgNVBAGMA25zdzEPMA0GA1UEBwwGc3lkbmV5
MREwDwYDVQQKAh0ZW1wIGNvbTEMMAAoGA1UECwwDaWN0MB4XDTEzMDkxMzEzNDgw
MloXDTE0MDkxMjEzNDgwMlowTTElMAkGA1UEBhMCNjExDDAKBgNVBAGMA25zdzEP
MA0GA1UEBwwGc3lkbmV5MREwDwYDVQQKAh0ZW1wIGNvbTEMMAAoGA1UECwwDaWN0
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQCChUH/yw0Ec9jsMLEntcJKUGBM5
3yHAMftI4aMqpWNT97Xa2A0KUcggKmOh/1oCVAP0FDzzYsV9oZhPsdIFZUd4PYHv
360CLFgAgOUErt5Act10m838Q5S7+vUm8pZP1FucSv/mAif0r8w2M4mVFILFyxUR
jpvxvy1bK2Uf0oP9+QIDAQABo1MwUTAdBgNVHQ4EFgQUUWoPQ0/Z08JraMAehRYu
bmdzEbMwHwYDVROjBBgwFoAUUWoPQ0/Z08JraMAehRYubmdzEbMwDwYDVROTAQH/
BAUwAwEB/zANBgkqhkiG9w0BAQsFAA0BgQCeRovl8mDd3pVMXhF9sgMLEBA+H3gA
17/L/9tnkG6TqZ6gioLkWH7WkojmLjJXHGQrgYJVGPo+nojCyAZ050ivIqEa2M2S
4e6wKN1rROi53yiOTrmSxM8viBBUfxnPLYx4HZ+6KmJHbBLomUdWPTmPuszeBxss
9Eda1hRR7zi51g==
-----END CERTIFICATE-----
anu@anu-Vostro-5471:~$
```

Self-signed certificates are not trusted by browsers by default — they are suitable for development and internal testing only. For production use, always obtain a certificate signed by a recognized CA.

Resources & Acknowledgments

Primary Resources

Cryptography and Network Security (Principles and Practice)
William Stallings — Chapter 13

Slides 1–33 of Lecture 13 from Avinash Kak's course:
<https://engineering.purdue.edu/kak/compsec/>

Acknowledgment

Lecture slides 39–49 are adapted from **Lecture 13 of Avinash Kak**,
Purdue University:

<https://engineering.purdue.edu/kak/compsec/>

Department of Computer Science & Engineering